

1 Adapted Algorithms

Algorithms 1, 2, and 3 show our adapted FHE algorithms.

Algorithm 1 Adapted Ray Casting Algorithm

Require: Point $P = (x, y)$, Polygon vertices $V = [v_0, v_1, \dots, v_{n-1}]$

Ensure: **true** if P is inside the polygon, **false** otherwise

```
1:  $edge\_result \leftarrow []$ 
2: for  $i \leftarrow 1$  to  $n - 1$  do
3:    $a \leftarrow V[i - 1]$ 
4:    $b \leftarrow V[i]$ 
5:   if  $a.y > b.y$  then ▷ Determine higher and lower vertex
6:      $v_1 \leftarrow a, v_2 \leftarrow b$ 
7:   else
8:      $v_1 \leftarrow b, v_2 \leftarrow a$ 
9:   end if
10:   $comp1 \leftarrow Comp(y, v_2.y)$  ▷ Is the ray within y-bounds of edge?
11:   $comp2 \leftarrow Comp(v_1.y, y)$ 
12:   $ybounds \leftarrow comp1 \cdot comp2$ 
13:   $orient \leftarrow (v_1.x - v_2.x) \cdot (y - v_2.y) - (v_1.y - v_2.y) \cdot (x - v_2.x)$  ▷ Is P on the right side of edge?
14:   $orient \leftarrow orientation \cdot \frac{1}{\|v_2 - v_1\|}$ 
15:   $orientSign \leftarrow Sign(orientation)$ 
16:
17:   $edge\_result.append(ybounds \cdot orientSign)$ 
18: end for
19: while  $len(edge\_result) > 1$  do ▷ Check if number of intersections is odd
20:    $temp \leftarrow []$ 
21:   for  $i \leftarrow 0$  to  $len(edge\_result) - 1$  step 2 do
22:     if  $i + 1 < len(edge\_result)$  then
23:        $temp.append(edge\_result[i] \oplus edge\_result[i + 1])$ 
24:     else
25:        $temp.append(edge\_result[i])$ 
26:     end if
27:   end for
28:    $edge\_result \leftarrow temp$ 
29: end while
30: return  $edge\_result[0]$ 
```

Algorithm 2 Adapted Winding Number Algorithm

Require: Point $P = (x, y)$, Polygon vertices $V = [v_0, v_1, \dots, v_{n-1}]$ **Ensure:** **true** if P is inside, **false** otherwise

```
1:  $angleSum \leftarrow 0$ 
2: for  $i = 1$  to  $n - 1$  do
3:    $v_1 \leftarrow V[i - 1]$ 
4:    $v_2 \leftarrow V[i]$ 
5:    $diff_1 \leftarrow v_1 - P$ 
6:    $diff_2 \leftarrow v_2 - P$ 
                                      $\triangleright$  Calculate dot product
7:    $dot \leftarrow diff_1 \cdot diff_2$ 
                                      $\triangleright$  Calculate distances between  $P$  and  $v_1 / v_2$ 
8:    $dist_1 \leftarrow \|diff_1\|$ 
9:    $dist_2 \leftarrow \|diff_2\|$ 
                                      $\triangleright$  Perform two divisions
10:   $denom_1 \leftarrow \frac{1}{dist_1}$ 
11:   $denom_2 \leftarrow \frac{1}{dist_2}$ 
                                      $\triangleright$  Calculate inverse cosine
12:   $\theta_i \leftarrow \arccos(dot \cdot denom_1 \cdot denom_2)$ 
13:   $angleSum \leftarrow angleSum + \theta_i$ 
14: end for
                                      $\triangleright$  Return true if the angles add up to  $360^\circ$ 
15: return  $Comp(angleSum, 2 \cdot \pi)$ 
```

Algorithm 3 Adapted Edge Orientation Algorithm

Require: Point $P = (x, y)$, Convex polygon vertices $V = [v_0, v_1, \dots, v_{n-1}]$ **Ensure:** **true** if P is inside, **false** otherwise

```
1: for  $i = 1$  to  $n - 1$  do
                                      $\triangleright$  Vertices are assumed to be in clockwise order
2:    $v_1 \leftarrow V[i - 1]$ 
3:    $v_2 \leftarrow V[i]$ 
4:    $det \leftarrow (v_1.x - v_2.x) \cdot (y - v_2.y) - (v_1.y - v_2.y) \cdot (x - v_2.x)$ 
                                      $\triangleright$  Is P on the right side of the edge?
5:    $count \leftarrow count + Sign(det)$ 
6: end for
7: return  $Comp(count, 0)$ 
```
